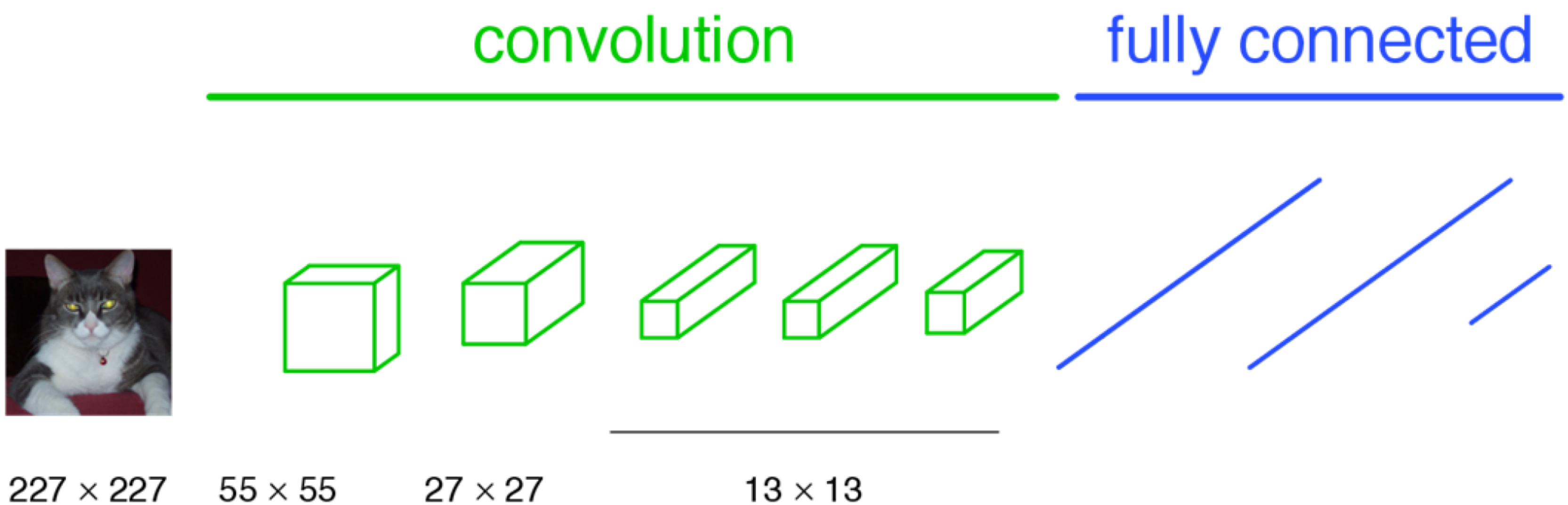
Namal University, Mianwali

Department of Computer Science

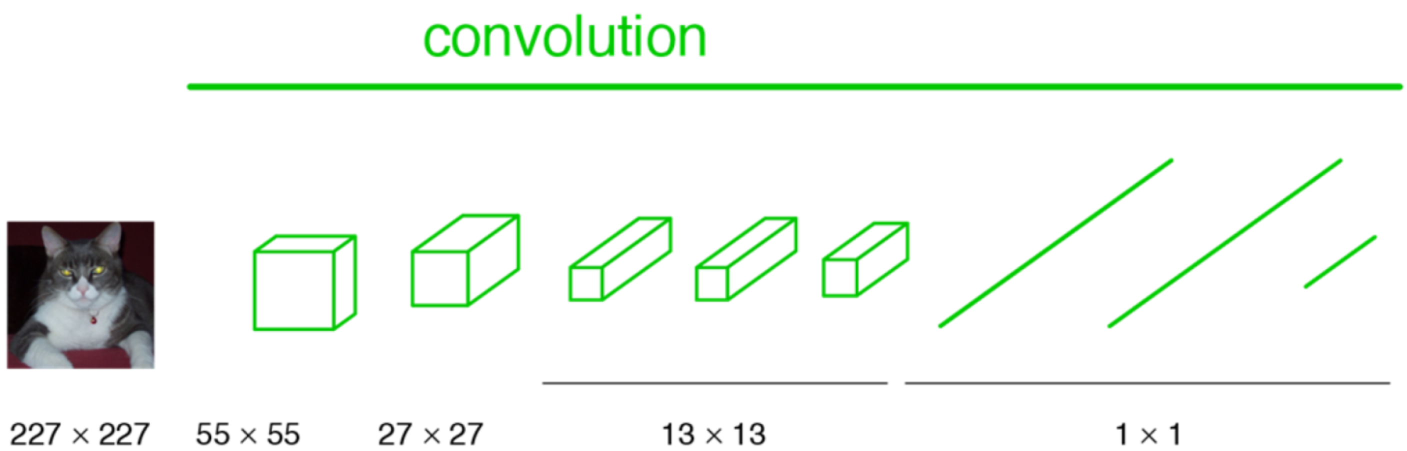
Machine Learning

Fully Convolutional Network

Classification Network

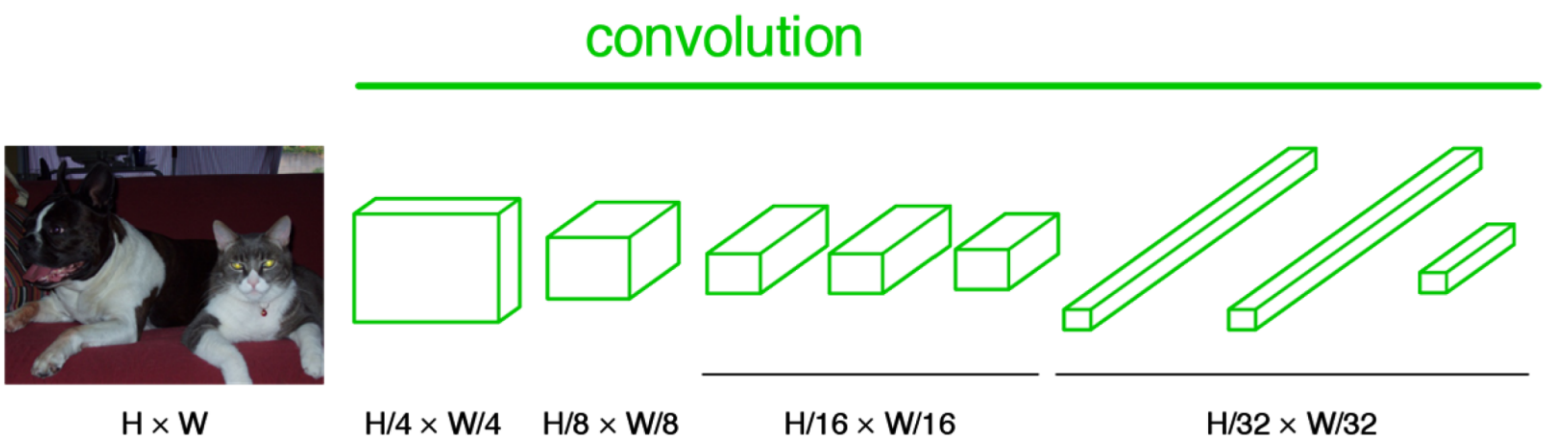


FCN: Becoming Fully Convolutional

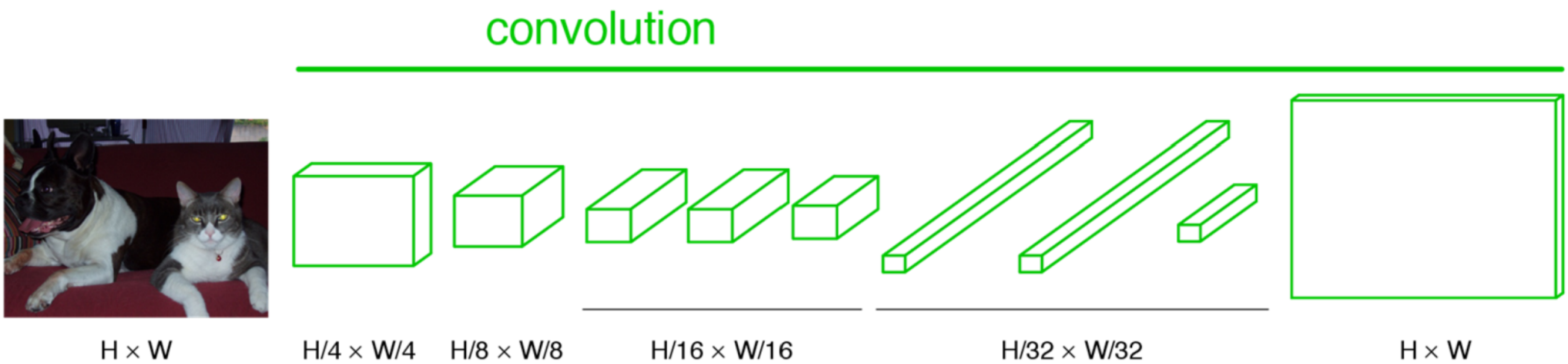


Convert fully connected layers to convolutional layers!

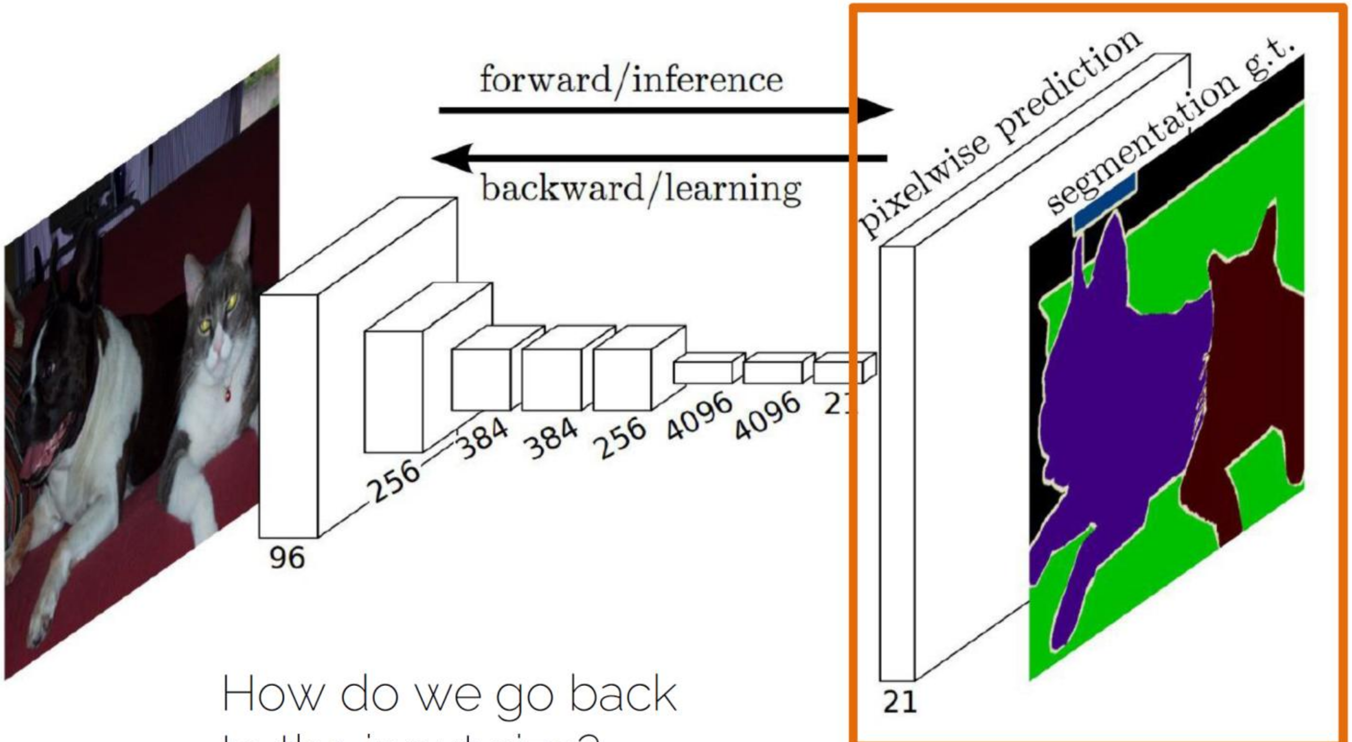
FCN: Becoming Fully Convolutional



FCN: Upsampling Output



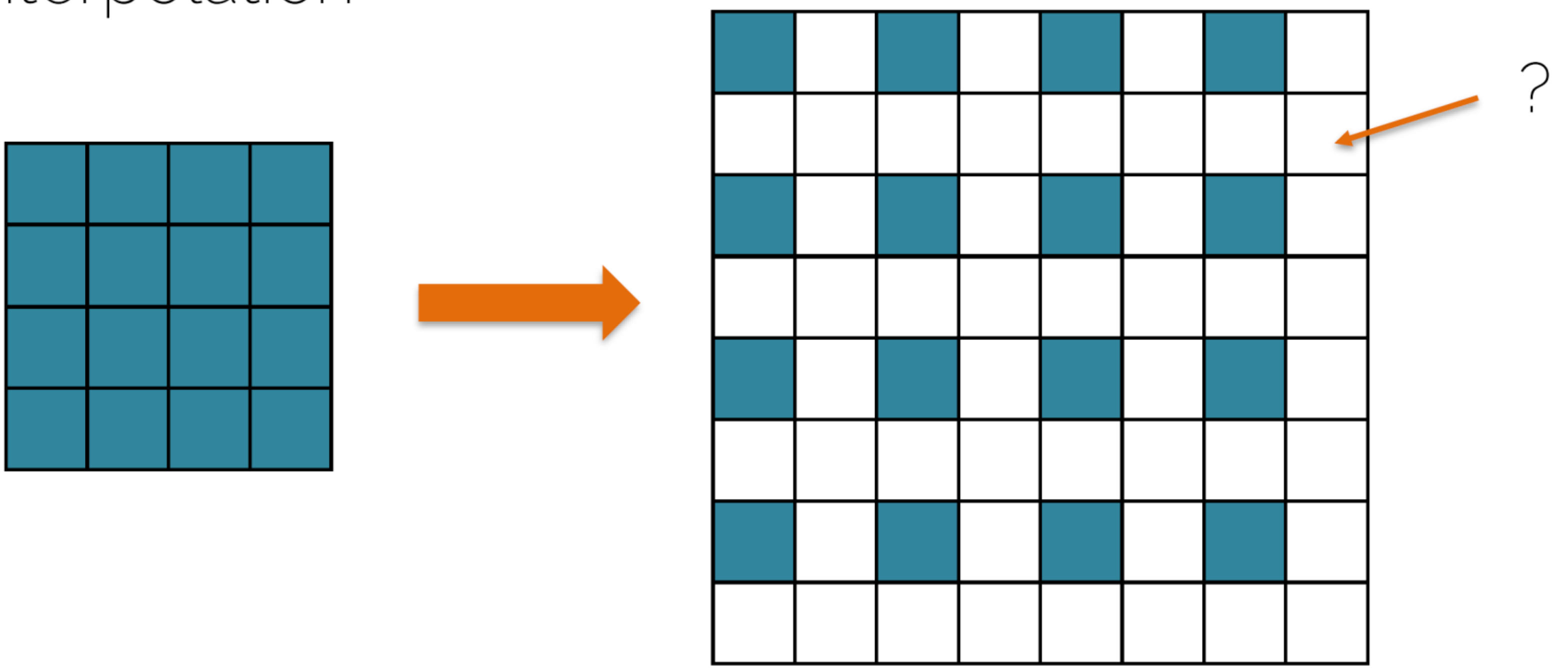
Semantic Segmentation (FCN)



How do we go back
to the input size?

Types of Upsampling

- 1. Interpolation



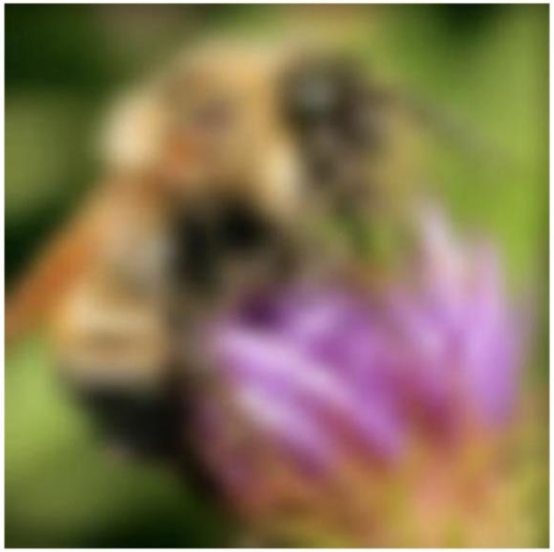
Types of Upsampling

- 1. Interpolation

Original image  **x 10**

Nearest neighbor interpolation

Bilinear interpolation



Bicubic interpolation

Nearest Neighbor Replacement

Simply Replicate the value from neighboring pixels.

1	0	1
1	1	0
1	0	1

1			0			1
1			1			0
1			0			1

Nearest Neighbor Replacement

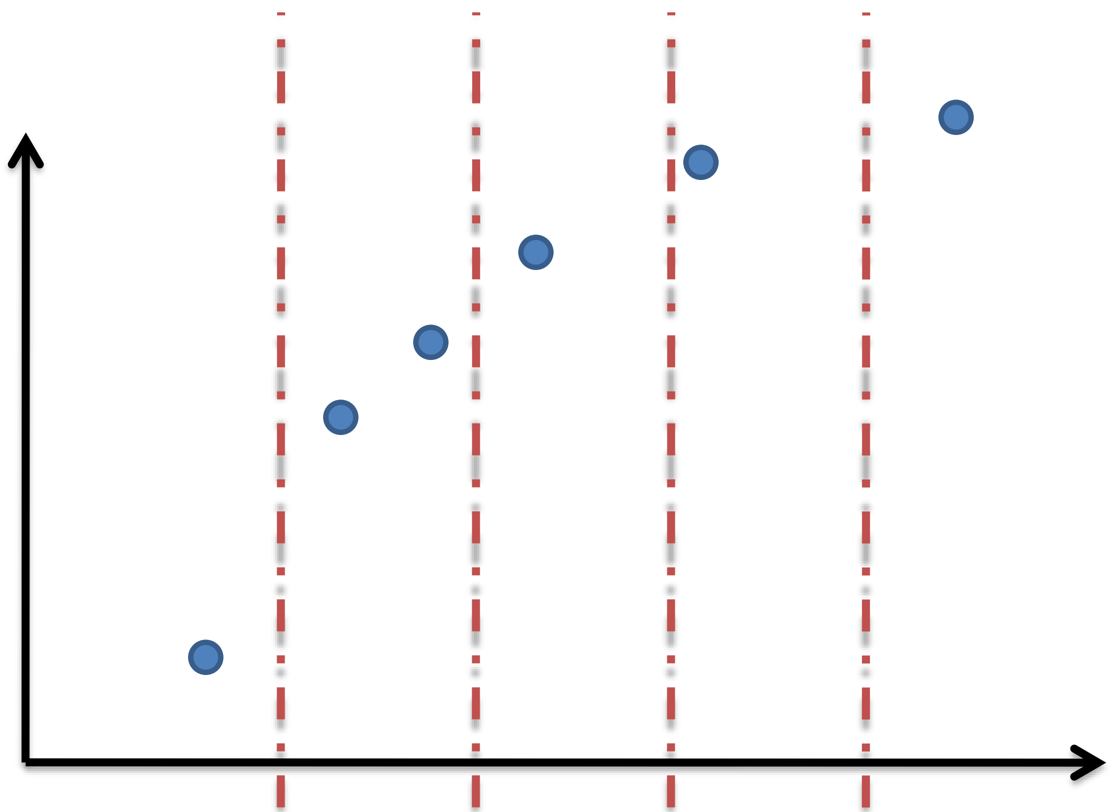
Adds blocky effect.

1	0	1
1	1	0
1	0	1

1	1	0	0	0	1	1
1	1	0	0	0	1	1
1	1	1	1	1	0	0
1	1	1	1	1	0	0
1	1	1	1	1	0	0
1	1	0	0	0	1	1
1	1	0	0	0	1	1

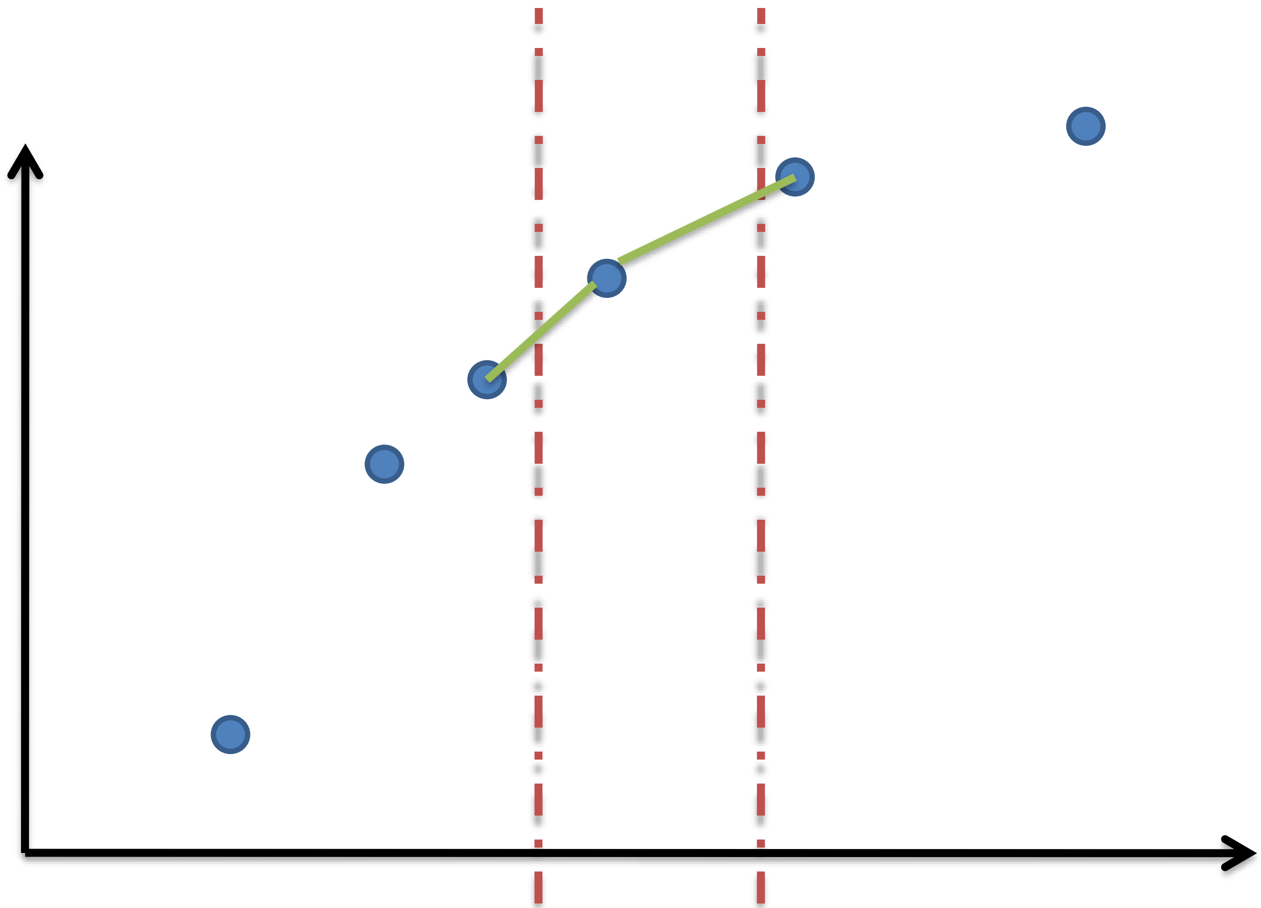
1-D Interpolation

Interpolation works by using known data to estimate values at unknown points.



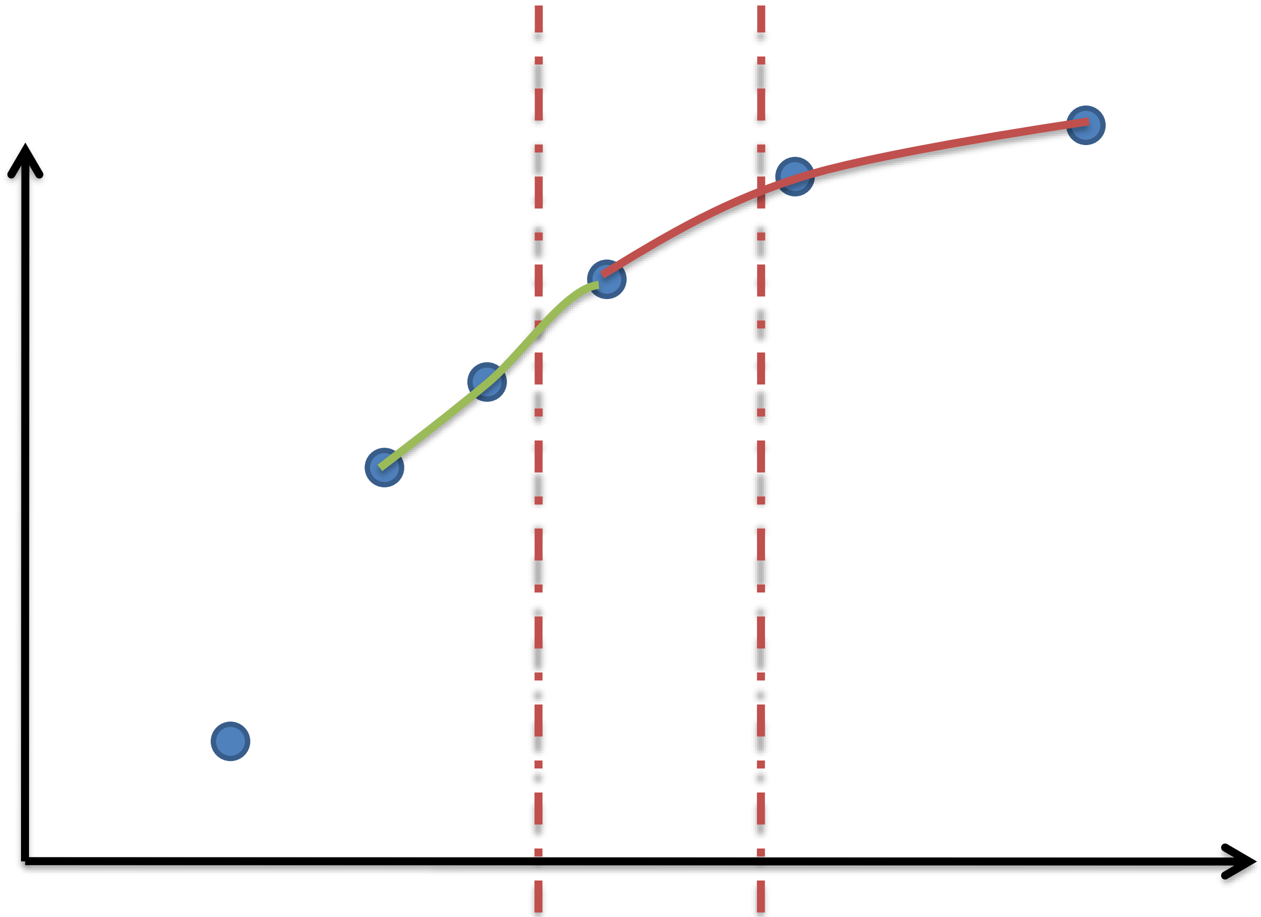
1-D Interpolation

Linear Interpolation: Fit a linear function piecewise between the points.



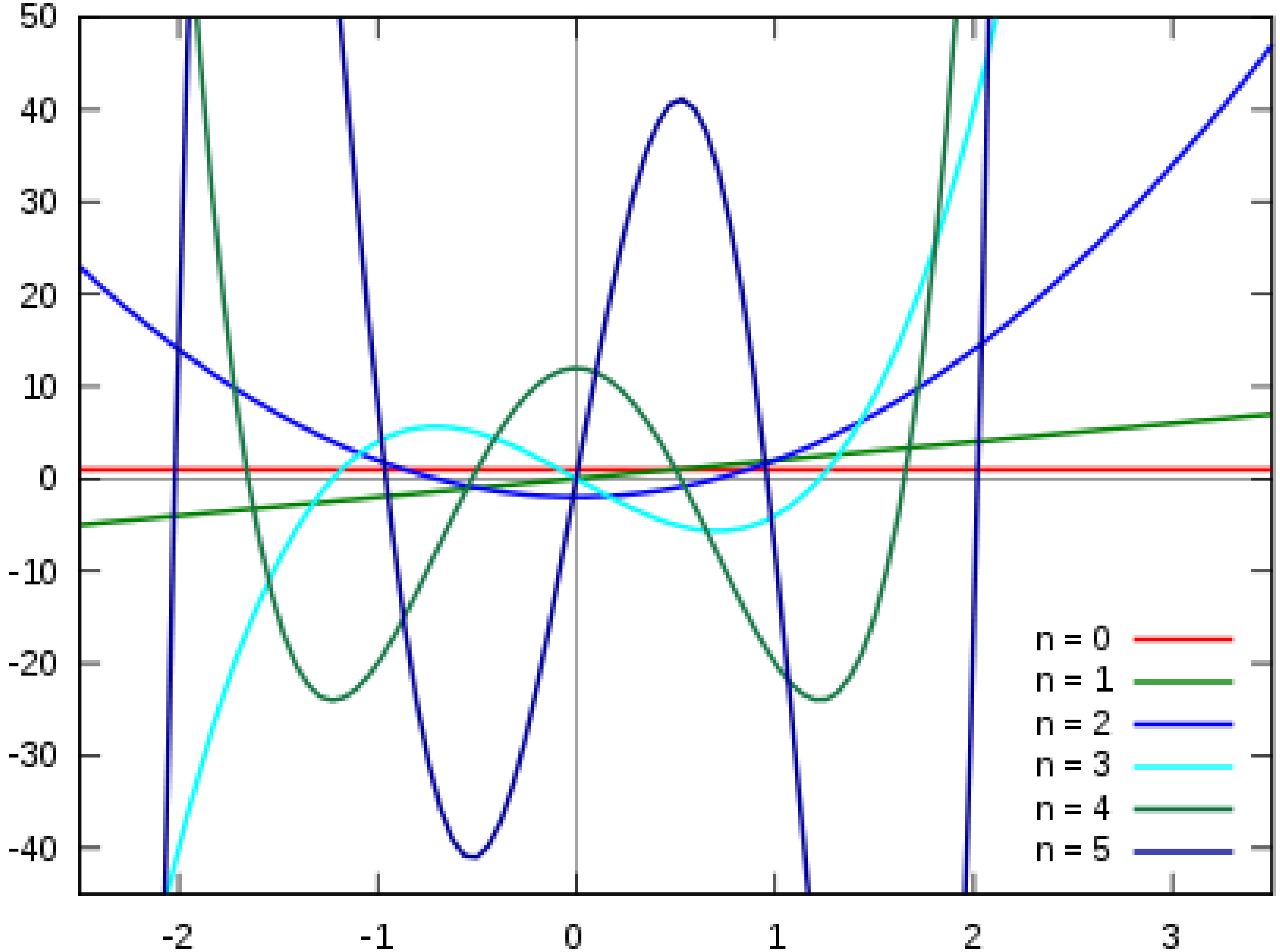
1-D Interpolation

Quadratic: Fit a Quadratic Polynomial between the three points.



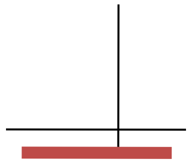
1-D Interpolation

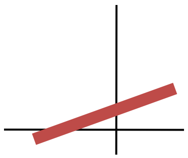
Fitting Polynomial to data points.

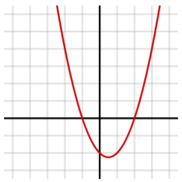


1-D Interpolation

Degree	Polynomial	How it looks	Req. Pts.
Zero Polynomial	$f(x) = 0$		None
0	$f(x) = a_0$	horizontal line with *y*-intercept a_0	1
1 (Linear)	$f(x) = a_0 + a_1 x$	an oblique line with *y*-intercept a_0 and slope a_1	2
2 (Quadratic)	$f(x) = a_0 + a_1 x + a_2 x^2$	Parabola	3
3 (Cubic)	$f(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3$	Spline	4







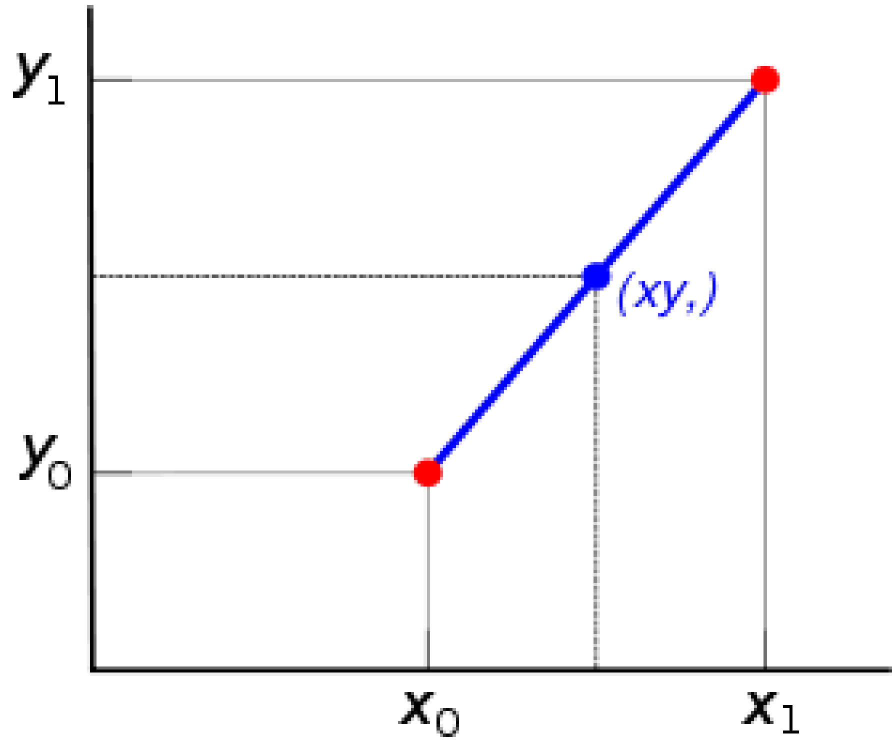
- No. of unknowns in a polynomial of degree n-1 = n
- Select n nearest known neighbours to generate n equations
- Solve the linear system of equations to estimate the constants
- Simply put the unknown x value and interpolate the y value

Linear Interpolation

- Equation of a line

$$\frac{y-y_0}{x-x_0}=\frac{y_1-y_0}{x_1-x_0}$$

$$y=\frac{y_1-y_0}{x_1-x_0}(x-x_0)+y_0$$

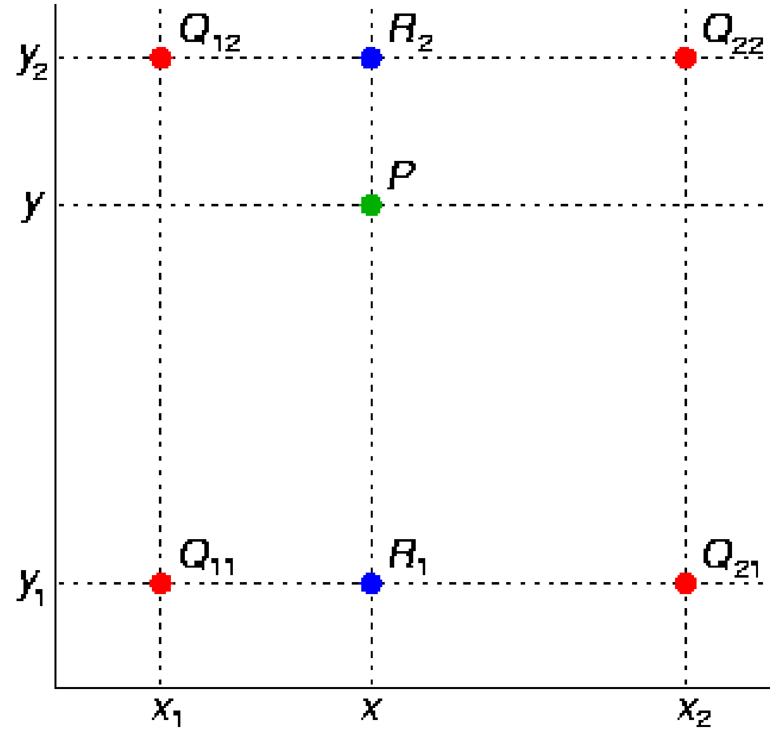


- Interpolation using Linear Polynomial
  - $y = a_0 + a_1x$
  - Requires two unknowns

Bi-linear Interpolation

- Key Points:
  - $Q_{11} = (x_1, y_1)$, $Q_{12} = (x_1, y_2)$,
  - $Q_{21} = (x_2, y_1)$, $Q_{22} = (x_2, y_2)$.

- Linear interpolation in the x-direction followed by linear interpolation in y-direction

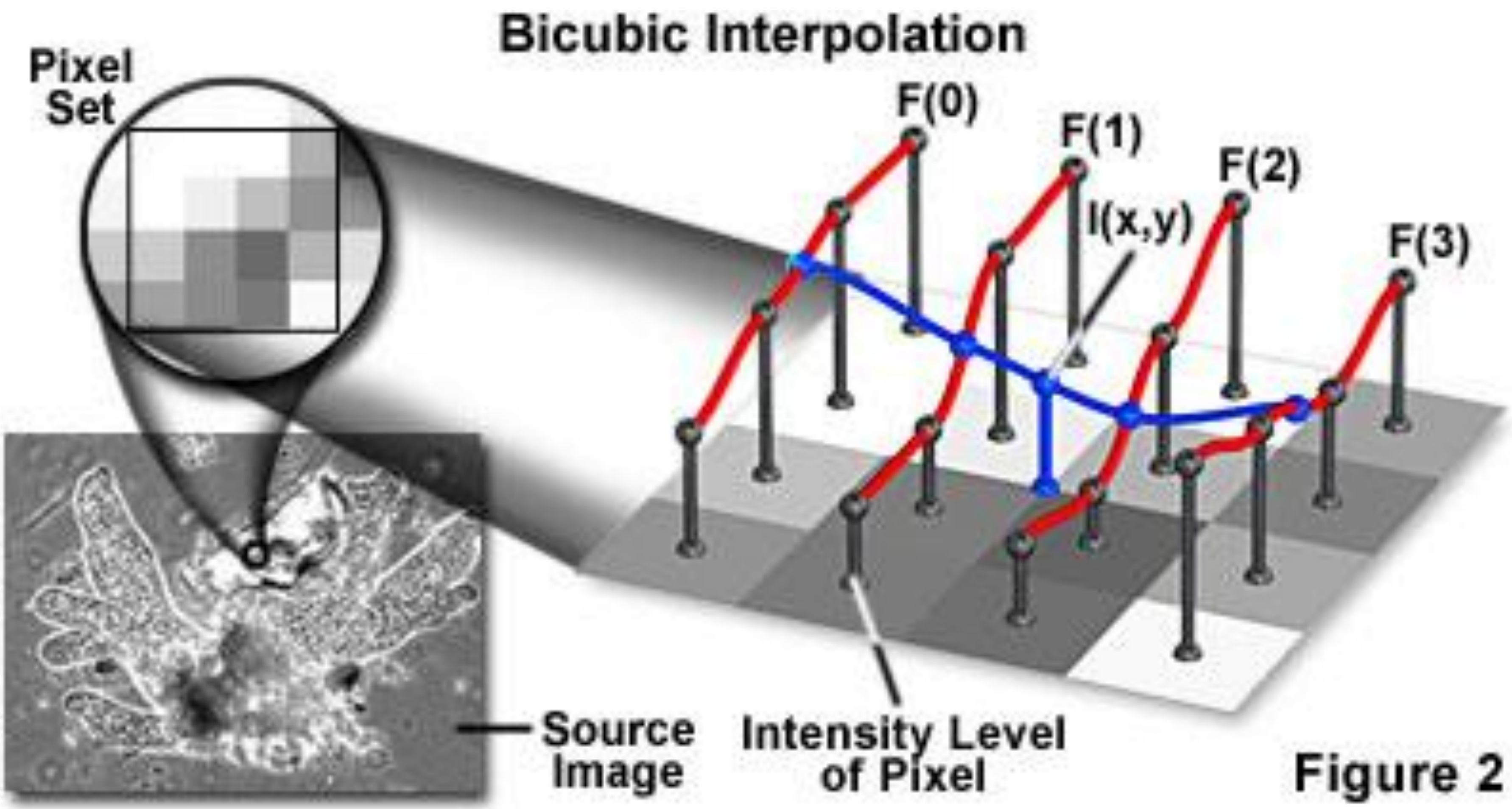


$$f(R_1) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21})$$

$$f(R_2) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22})$$

$$f(P) \approx \frac{y_2 - y}{y_2 - y_1} f(R_1) + \frac{y - y_1}{y_2 - y_1} f(R_2).$$

Interpolation



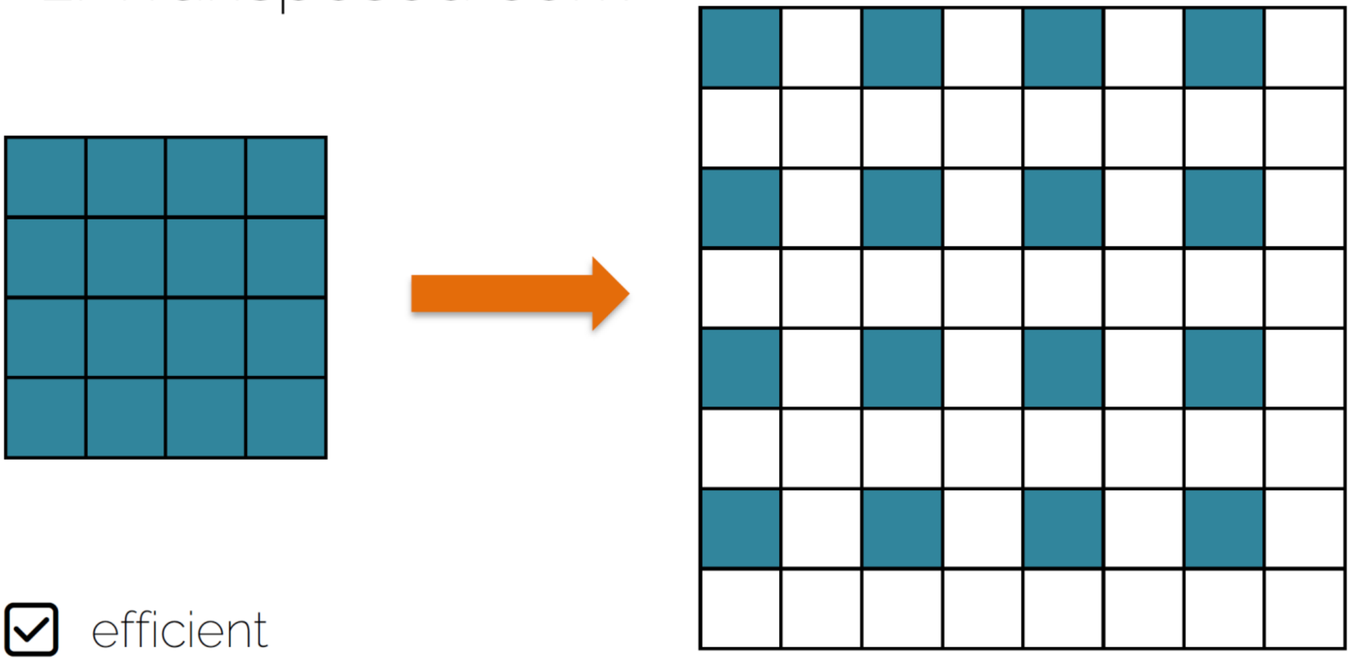
Types of Upsampling

- 1. Interpolation

  ☑ Few artifacts

Types of Upsampling

- 2. Transposed conv

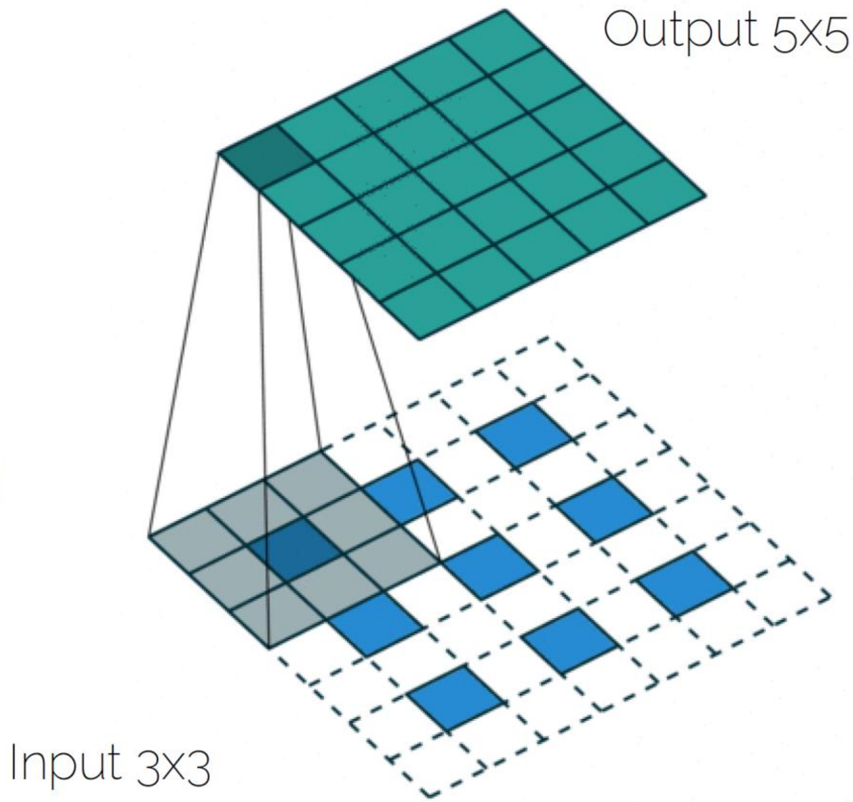


+ CONVS

☑ efficient

Types of Upsampling

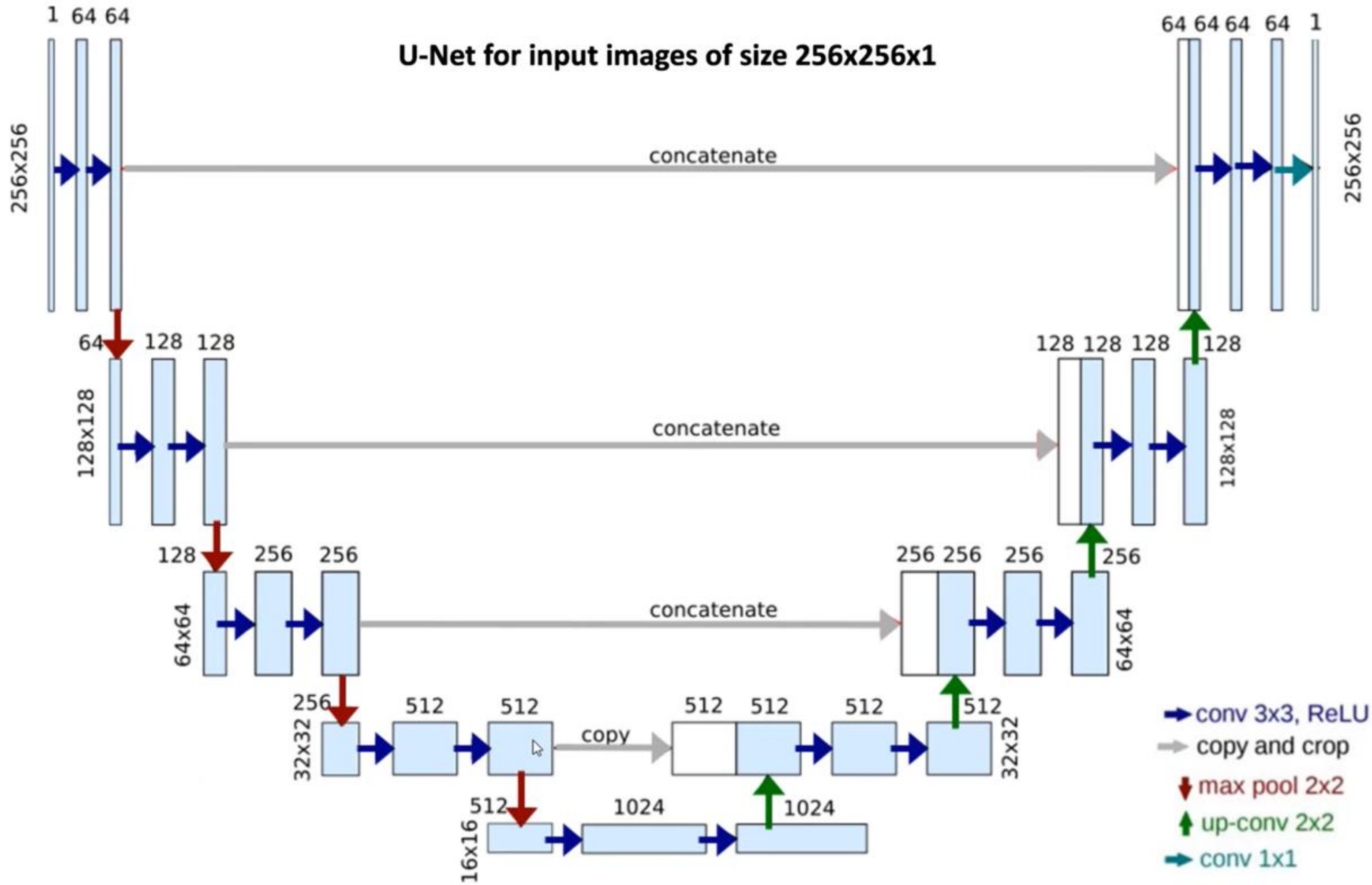
- 2. Transposed convolution
  - Unpooling
  - Convolution filter (learned)
  - Also called up-convolution (**never** deconvolution)



Refined Outputs

- If one does a cascade of unpooling + conv operations, we get to the encoder-decoder architecture

- Even more refined: Autoencoders with skip connections (aka U-Net)


U-Net for input images of size 256x256x1
1 64 64
256x256
concatenate
64 64 64 64 1
256x256
64 128 128
128x128
concatenate
128 128 128 128
128x128
128 256 256
64x64
concatenate
256 256 256 256
64x64
256 512 512
32x32
copy
512 512 512 512
32x32
512 1024 1024
16x16
conv 3x3, ReLU
copy and crop
max pool 2x2
up-conv 2x2
conv 1x1

U-Net: Encoder

Left side: **Contraction Path** (Encoder)

- Captures context of the image
- Follows typical architecture of a CNN:
  - Repeated application of 2 unpadded 3x3 convolutions
  - Each followed by ReLU activation
  - 2x2 maxpooling operation with stride 2 for downsampling
  - At each downsampling step, # of channels is doubled

→ as before: Height, Width ↓, Depth: ↑

U-Net: Decoder

Right Side: **Expansion Path** (Decoder):

- Upsampling to recover spatial locations for assigning class labels to each pixel
  - 2x2 up-convolution that halves number of input channels
  - **Skip Connections**: outputs of up-convolutions are concatenated with feature maps from encoder
  - Followed by 2 ordinary 3x3 convs
  - final layer: 1x1 conv to map 64 channels to # classes

- Height, Width: ↑ ,   Depth: ↓

Thank You